

Keenon Robot Interface Customization Project

Project Code: KEENON-RAG-INTERFACE

Client: Waaree Autonova

Date: March 11, 2026

Developer Assignment Document

Executive Summary

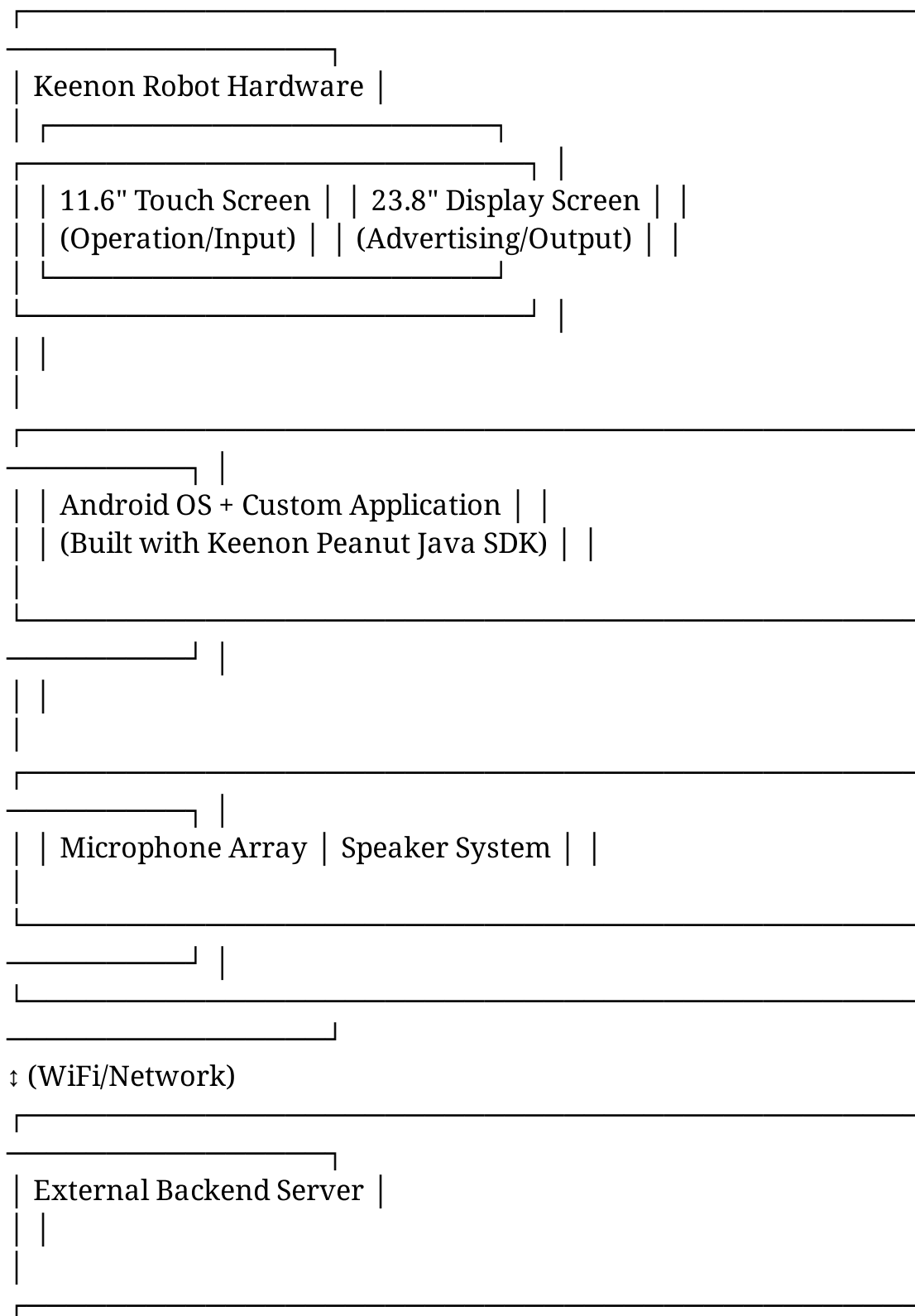
This project involves customizing a Keenon delivery robot using the Java SDK (codename "Peanut") to create an intelligent conversational interface. The robot will accept user inputs through forms and voice commands, process them via an external RAG (Retrieval-Augmented Generation) backend, and display responses on the robot's dual-display system.

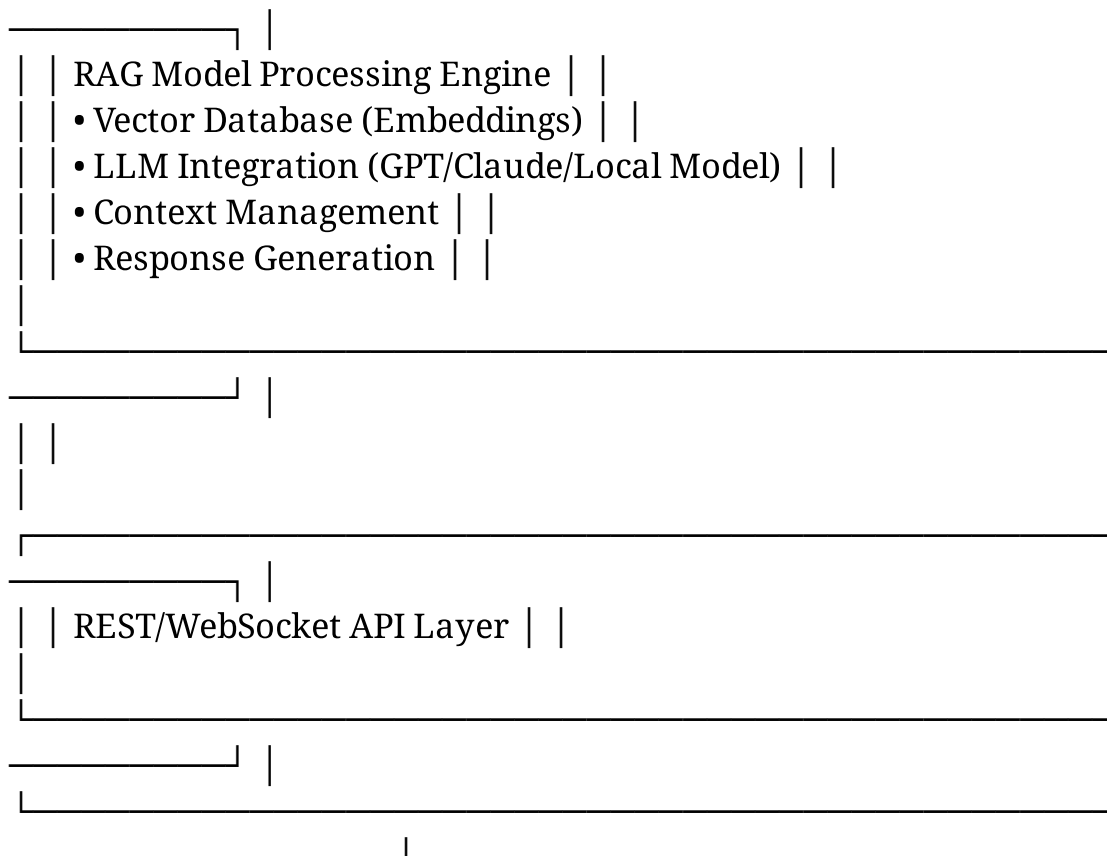
Project Objectives

- Develop custom Android application using Keenon Java SDK (Peanut) to replace/extend default PEANUT app functionality
- Implement dual input methods: touch-based forms and voice commands
- Integrate external backend running RAG model for intelligent conversation processing
- Manage and control both robot displays (touchscreen and non-touch advertising display)
- Create seamless conversational experience through robot interface

System Architecture

High-Level Architecture





Component Breakdown

Robot-Side Components:

1. Custom Android Application

- Built using Keenon Peanut Java SDK
- Manages both input methods (touch and voice)
- Handles display rendering on both screens
- Maintains conversation state and session management

2. Input Processing Module

- Form-based input handler for touch screen
- Voice recognition integration (Android Speech Recognition API)
- Input validation and preprocessing

3. Display Management Module

- Touch screen controller (11.6" - 1080P operation screen)
- Advertising screen controller (23.8" - 1080P display screen)
- Dynamic content rendering based on conversation flow

4. Network Communication Module

- WebSocket/REST client for backend communication

- Request/response queue management
- Error handling and retry logic
- Network status monitoring

Backend Components:

1. RAG Processing Engine

- Vector database for document embeddings
- Context retrieval system
- LLM integration for response generation
- Conversation history management

2. API Server

- RESTful endpoints for synchronous requests
- WebSocket support for real-time bidirectional communication
- Authentication and session management
- Rate limiting and load balancing

Technical Requirements

Robot Hardware Specifications

Based on Keenon T-Series robots (T8/T9/T10 variants)[1][2]:

Component	Specification
Touch Screen	11.6 inch, 1080P resolution
Display Screen	23.8 inch, 1080P resolution
Operating System	Android (customizable)
Network	WiFi (2412-2472MHz)
Sensors	LiDAR, Stereo Vision, VSLAM
Dimensions (W×D×H)	485.7 × 554.5 × 1398.8mm
Weight	Approximately 58 Kg

Table 1: Keenon Robot Hardware Specifications

Software Stack Requirements

Robot Application:

- Language: Java (Android development)
- SDK: Keenon Peanut Java SDK
- Minimum Android Version: Android 7.0 (API Level 24) or higher
- Build Tool: Gradle
- UI Framework: Android XML layouts or Jetpack Compose (if supported)

Required Android Permissions:

Backend Server:

- Framework: Python (FastAPI/Flask) or Node.js (Express)
- RAG Stack: LangChain or LlamaIndex
- Vector Database: Pinecone, Weaviate, ChromaDB, or FAISS
- LLM: OpenAI GPT-4, Anthropic Claude, or self-hosted model (Llama, Mistral)
- WebSocket Support: [Socket.IO](#) or native WebSocket implementation

Functional Requirements

FR1: User Input via Touch Interface

Description: Users can interact with the robot through form-based inputs on the touchscreen.

Implementation Details:

1. Create custom UI layouts for data collection
2. Support various input types:
 - Text input fields
 - Selection dropdowns
 - Radio buttons/checkboxes
 - Number pickers
 - Date/time pickers
3. Implement input validation before sending to backend
4. Provide visual feedback during processing

5. Display results on appropriate screen(s)

User Flow:

User approaches robot → Welcome screen displays on both screens

- User taps touchscreen → Input form appears
- User fills form → Submits → Loading indicator
- Backend processes via RAG → Response displayed
- Continue conversation or return to home

FR2: User Input via Voice Commands

Description: Users can interact with the robot using natural voice commands[3].

Implementation Details:

1. Integrate Android Speech Recognition API
(android.speech.SpeechRecognizer)
2. Implement wake word detection or push-to-talk mechanism
3. Voice activity detection (VAD) to detect when user stops speaking
4. Convert speech to text
5. Send transcribed text to backend RAG model
6. Convert text response back to speech (Text-to-Speech)
7. Play audio response through robot speakers

Voice Interaction Flow:

Wake word detected / Button pressed

- Microphone activates (visual indicator on screen)
- User speaks query
- Speech-to-text conversion
- Text sent to backend
- RAG processes query
- Text response received
- Text-to-speech conversion
- Audio played + text displayed on screen

Recommended Libraries:

- Speech Recognition: Android native SpeechRecognizer or Google Cloud Speech-to-Text

- Text-to-Speech: Android native TTS or Google Cloud Text-to-Speech
- Wake Word Detection: Porcupine Wake Word Engine or Snowboy

FR3: External RAG Backend Integration

Description: All user queries (form or voice) are processed by external backend running RAG model[4][5].

Backend Responsibilities:

1. Receive user query via REST/WebSocket
2. Retrieve relevant context from vector database
3. Generate response using LLM with retrieved context
4. Return structured response to robot
5. Maintain conversation history for context continuity

API Contract Example:

Request (from robot to backend):

```
{
  "session_id": "uuid-12345",
  "user_query": "What are today's special menu items?",
  "input_type": "voice",
  "timestamp": "2026-03-11T19:40:00Z",
  "conversation_history": [
    {
      "role": "user",
      "content": "Hello",
      "timestamp": "2026-03-11T19:35:00Z"
    },
    {
      "role": "assistant",
      "content": "Welcome! How can I help you today?",
      "timestamp": "2026-03-11T19:35:02Z"
    }
  ]
}
```

Response (from backend to robot):

```
{
  "session_id": "uuid-12345",
  "response_text": "Today's specials include Butter Chicken, Paneer Tikka Masala, and Vegetable Biryani. Would you like to know more about any of these?",
  "display_data": {
    "type": "menu_items",
    "items": [
      {
        "name": "Butter Chicken",
        "price": "₹350",
        "image_url": "https://..."
      },
      {
        "name": "Paneer Tikka Masala",
        "price": "₹280",
        "image_url": "https://..."
      }
    ]
  },
  "suggested_actions": [
    "Tell me more about Butter Chicken",
    "Show me dessert options",
    "I'm ready to order"
  ],
  "tts_audio_url": "https://backend.com/audio/response\_12345.mp3",
  "timestamp": "2026-03-11T19:40:03Z"
}
```

Communication Protocol:

- Use WebSocket for real-time bidirectional communication (preferred)
- Fallback to REST API with polling for simpler implementation
- Implement heartbeat mechanism to maintain connection
- Handle reconnection logic with exponential backoff

FR4: Dual Display Management

Description: Control content on both robot displays simultaneously based on conversation context.

Display Strategy:

Screen	Primary Use	Content Examples
Touch Screen (11.6")	User interaction, input forms, conversation interface	- Input forms - Conversation transcript - Action buttons - Keyboard/input methods - Status indicators
Display Screen (23.8")	Visual output, rich media display, advertising	- Product images/videos - Menu items with images - Promotional content - Visual responses - Branding materials - Maps/directions

Table 2: Dual Display Content Strategy

Implementation Approach:

1. Use Android Presentation API for secondary display management
2. Implement custom display manager class to coordinate both screens
3. Synchronize content updates based on conversation state
4. Support different display modes:
 - Idle mode: Branding/advertising on both screens
 - Active conversation: Interactive on touch screen, rich media on display screen
 - Information display: Synchronized content across both screens

Code Structure Example:

```
public class DualDisplayManager {  
    private DisplayManager displayManager;  
    private Display touchDisplay; // 11.6" interactive  
    private Display advertisingDisplay; // 23.8" non-touch  
  
    public void updateTouchScreen(View contentView) {  
        // Update touch screen content  
    }  
  
    public void updateAdvertisingScreen(Presentation presentation) {  
        // Update advertising screen content  
    }  
  
    public void syncDisplays(ConversationState state) {  
        // Coordinate content across both displays  
    }  
}
```

FR5: Conversation State Management

Description: Maintain conversation context throughout user interaction session.

Requirements:

- Generate unique session ID for each conversation
- Store conversation history locally on robot (temporary)
- Send conversation context with each backend request
- Implement session timeout (e.g., 5 minutes of inactivity)
- Clear session data after timeout or explicit user end
- Handle multiple concurrent sessions if robot supports multi-user interaction

Non-Functional Requirements

NFR1: Performance

- Voice recognition latency: $< 500\text{ms}$
- Backend response time: < 2 seconds for typical queries
- Display update latency: $< 100\text{ms}$
- Support continuous conversation without lag

NFR2: Reliability

- Handle network disconnections gracefully with retry logic
- Implement local fallback responses for common queries
- Auto-recovery from application crashes
- Maintain conversation state through temporary connection loss

NFR3: Usability

- Clear visual indicators for voice recording status
- Intuitive touch interface with large, accessible buttons
- Multi-language support (minimum: English, Hindi)
- Accessibility features for users with disabilities

NFR4: Security

- Encrypt all communication between robot and backend (TLS/SSL)
- Implement API authentication (JWT tokens or API keys)
- Do not store sensitive user data on robot beyond session
- Comply with data privacy regulations

Development Phases

Phase 1: Foundation Setup (Week 1-2)

Deliverables:

1. Keenon SDK integration and basic app structure
2. Dual display initialization and basic rendering
3. Network communication module (REST/WebSocket client)
4. Backend API server setup with basic RAG pipeline

Tasks:

- Set up Android development environment with Keenon SDK
- Create basic application extending/replacing PEANUT app
- Implement display detection and management
- Set up backend server with basic LLM integration
- Create API endpoints for conversation handling
- Test basic communication between robot and backend

Phase 2: Input Methods Implementation (Week 3-4)**Deliverables:**

1. Touch-based form input system
2. Voice recognition integration
3. Input validation and preprocessing
4. Visual feedback for both input modes

Tasks:

- Design and implement form UI layouts
- Integrate Android Speech Recognition
- Implement wake word or push-to-talk mechanism
- Create input validation logic
- Build visual indicators for voice recording
- Test both input methods end-to-end

Phase 3: RAG Backend Integration (Week 5-6)**Deliverables:**

1. Fully functional RAG pipeline on backend
2. Vector database with initial knowledge base
3. Conversation context management
4. API integration with robot application

Tasks:

- Set up vector database and populate with domain knowledge
- Implement RAG retrieval and generation pipeline
- Create conversation history management
- Optimize prompt engineering for quality responses

- Implement error handling and fallback mechanisms
- Load testing and performance optimization

Phase 4: Display Content Management (Week 7-8)

Deliverables:

1. Synchronized dual-display rendering
2. Rich media content display system
3. Dynamic content updates based on conversation
4. Idle mode with advertising/branding

Tasks:

- Implement Presentation API for secondary display
- Create display coordination logic
- Build content rendering system for various data types
- Design and implement idle mode screens
- Create smooth transitions between display states
- Test display synchronization

Phase 5: Testing and Refinement (Week 9-10)

Deliverables:

1. Fully tested application with all features integrated
2. Performance optimization completed
3. Bug fixes and edge case handling
4. Documentation and deployment guide

Tasks:

- End-to-end testing of all features
- User acceptance testing with real scenarios
- Performance profiling and optimization
- Security testing and hardening
- Create deployment documentation
- Prepare training materials for operators

Technical Architecture Details

Robot Application Architecture

Package Structure:

com.waaree.keenon.interface/

- |— MainActivity.java // Main entry point
- |— ui/
 - |— TouchScreenActivity.java // Touch screen UI controller
 - |— DisplayPresentation.java // Secondary display controller
 - |— FormInputFragment.java // Form-based input
 - |— ConversationFragment.java // Conversation display
- |— input/
 - |— VoiceInputManager.java // Voice recognition handler
 - |— FormInputHandler.java // Form input handler
 - |— InputValidator.java // Input validation
- |— display/
 - |— DualDisplayManager.java // Display coordination
 - |— ContentRenderer.java // Dynamic content rendering
 - |— ScreenStateManager.java // Screen state transitions
- |— network/
 - |— BackendClient.java // API client
 - |— WebSocketHandler.java // WebSocket communication
 - |— NetworkStateMonitor.java // Network status tracking
- |— models/
 - |— ConversationMessage.java // Message data model
 - |— BackendRequest.java // Request model
 - |— BackendResponse.java // Response model
- |— utils/
 - |— SessionManager.java // Session management
 - |— AudioPlayer.java // TTS audio playback
 - |— Logger.java // Logging utility
- |— KeenSDKWrapper.java // Keenon SDK integration layer

Backend Architecture

Recommended Stack (Python + FastAPI):

```
backend/
├── main.py // FastAPI application entry
├── api/
│   ├── routes.py // API endpoints
│   └── websocket.py // WebSocket handlers
├── rag/
│   ├── retriever.py // Vector DB retrieval
│   ├── generator.py // LLM response generation
│   └── embeddings.py // Embedding creation
├── models/
│   ├── conversation.py // Conversation models
│   └── schema.py // Pydantic schemas
├── services/
│   ├── session_service.py // Session management
│   └── tts_service.py // Text-to-speech generation
├── database/
│   ├── vector_db.py // Vector DB interface
│   └── session_store.py // Session storage (Redis)
├── config/
│   └── settings.py // Configuration
└── requirements.txt // Python dependencies
```

Data Flow Sequence

Voice Input Flow:

User speaks → Microphone captures audio
→ Android SpeechRecognizer converts to text
→ Text sent to backend via WebSocket
→ Backend retrieves relevant context from vector DB
→ LLM generates response with context
→ Response sent back to robot
→ Robot displays text on touch screen
→ Robot shows rich media on display screen
→ TTS converts response to audio
→ Audio plays through speakers

Form Input Flow:

User fills form on touch screen

- Form data validated locally
- Structured data sent to backend via REST API
- Backend processes form data
- RAG generates contextual response
- Response includes display directives
- Touch screen shows confirmation/next steps
- Display screen shows relevant visual content

Integration with Keenon SDK

SDK Capabilities to Leverage

Based on Keenon robot capabilities[1]:

1. Navigation System

- SLAM-based navigation
- Obstacle avoidance
- Path planning

Usage: Can integrate conversational commands for robot navigation ("Take me to Table 5")

2. Guardian Mode

- Auto-restart application if closed
- Ensures custom app stays running

Usage: Enable to prevent accidental app closure

3. System Settings Access

- Boot-up and self-start configuration
- Network settings

Usage: Configure app to auto-start on robot boot

SDK Integration Points

Key SDK Methods to Implement:

```
// Initialize Keenon SDK
```

```
KeenSDK.initialize(context);
```

```
// Access robot navigation
```

```
RobotNavigation navigation = KeenSDK.getNavigation();
```

```
// Control robot movement (if needed)
navigation.moveTo(destinationId);

// Access display management
DisplayController displayController =
KeenSDK.getDisplayController();

// Set up guardian mode for app persistence
KeenSDK.setGuardianMode(true);

// Configure boot-up auto-start
KeenSDK.setAutoStart(true);
```

Note: Exact SDK API details should be obtained from Keenon's official documentation. The above is illustrative.

RAG Model Implementation Guide

RAG Architecture Components

1. Document Processing

- Ingest domain-specific documents (menus, FAQs, policies, etc.)
- Split documents into chunks (500-1000 tokens)
- Generate embeddings using embedding model
- Store embeddings in vector database

2. Retrieval Pipeline

- Convert user query to embedding
- Perform similarity search in vector DB
- Retrieve top-k most relevant chunks (k=3-5)
- Re-rank results for relevance

3. Generation Pipeline

- Construct prompt with retrieved context
- Include conversation history for continuity
- Generate response using LLM
- Post-process response for display formatting

Sample RAG Implementation (Python)

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationalRetrievalChain
```

```
class RAGEngine:
```

```
    def init(self):
```

```
        # Initialize embedding model
```

```
        self.embeddings = OpenAIEmbeddings()
```

```
        # Initialize vector store
```

```
        self.vectorstore = Chroma(
            embedding_function=self.embeddings,
            persist_directory="./vector_db"
        )
```

```
        # Initialize LLM
```

```
        self.llm = ChatOpenAI(
            model="gpt-4",
            temperature=0.7
        )
```

```
        # Create retrieval chain
```

```
        self.qa_chain = ConversationalRetrievalChain.from_llm(
            llm=self.llm,
            retriever=self.vectorstore.as_retriever(
                search_kwargs={"k": 5}
            )
        )
```

```
    def process_query(self, query, conversation_history):
```

```
        """Process user query with RAG"""
```

```
        response = self.qa_chain({
            "question": query,
            "chat_history": conversation_history
        })
```

```
return {
  "answer": response["answer"],
  "source_documents": response.get("source_documents", [])
}
```

Vector Database Options

Database	Pros	Cons
Pinecone	Managed service, scalable, fast	Requires subscription
Weaviate	Open-source, feature-rich	Requires self-hosting
ChromaDB	Simple, lightweight, local	Limited scalability
FAISS	Fast, local, free	No built-in persistence

Table 3: Vector Database Comparison

Recommendation: Start with ChromaDB for development and prototype. Move to Pinecone or Weaviate for production deployment.

API Specifications

REST API Endpoints

Base URL: `https://api.yourbackend.com/v1`

1. Create Conversation Session

POST /sessions

Request:

```
{
  "robot_id": "keenon_001",
```

```
"location": "restaurant_hall_a"
}
```

Response:

```
{
"session_id": "uuid-12345",
"created_at": "2026-03-11T19:40:00Z",
"expires_at": "2026-03-11T19:45:00Z"
}
```

2. Process Query

POST /query

Request: (See FR3 for detailed example)

Response: (See FR3 for detailed example)

3. End Session

DELETE /sessions/{session_id}

Response:

```
{
"status": "session_ended",
"session_id": "uuid-12345"
}
```

WebSocket Protocol

Connection URL: wss://api.yourbackend.com/v1/ws

Connection Flow:

1. Client connects to WebSocket
2. Client sends authentication message
3. Server confirms authentication
4. Client sends queries, server responds in real-time
5. Client sends heartbeat every 30 seconds
6. Server responds with pong
7. Connection maintained until explicit close or timeout

Message Format:

```
{  
  "type": "query | response | heartbeat | pong | error",  
  "session_id": "uuid-12345",  
  "payload": { ... },  
  "timestamp": "2026-03-11T19:40:00Z"  
}
```

Testing Strategy

Unit Testing

- Test individual components in isolation
- Mock external dependencies (SDK, network)
- Target: 80% code coverage minimum

Key Areas:

- Input validation logic
- Display coordination logic
- Session management
- Network request/response handling

Integration Testing

- Test communication between robot app and backend
- Test dual display synchronization
- Test voice-to-backend-to-response flow
- Test form-to-backend-to-response flow

End-to-End Testing

Test Scenarios:

1. Happy Path - Voice Interaction

- User approaches robot
- Speaks query
- Receives response on both displays
- Continues conversation
- Ends session naturally

2. Happy Path - Form Interaction

- User taps screen

- Fills form
- Submits
- Receives response

3. Network Failure Recovery

- Conversation in progress
- Network disconnects
- Robot shows error state
- Network reconnects
- Conversation resumes

4. Voice Recognition Failure

- User speaks unclear query
- Recognition fails
- Robot prompts user to repeat
- Successful on retry

5. Session Timeout

- User starts conversation
- Leaves robot idle for 5+ minutes
- Session expires
- Robot returns to idle state

Deployment Guide

Robot Application Deployment

1. Build APK using Android Studio
2. Sign APK with release keystore
3. Transfer APK to robot via USB or network
4. Install using ADB: `adb install app-release.apk`
5. Configure Guardian Mode to auto-restart app
6. Configure Boot-up Self-Start for automatic launch
7. Test application on robot hardware

Backend Deployment

Option 1: Cloud Deployment (Recommended for Production)

- Deploy on AWS, GCP, or Azure
- Use containerization (Docker)
- Set up load balancer for scalability
- Configure auto-scaling based on load

- Use managed vector database service

Option 2: On-Premise Deployment

- Deploy on local server in facility
- Ensures data privacy and low latency
- Requires server maintenance and monitoring
- Set up reverse proxy (Nginx) for SSL termination

Deployment Checklist:

- ☐ Environment variables configured
- ☐ SSL certificates installed
- ☐ Vector database populated with knowledge base
- ☐ LLM API keys configured and validated
- ☐ Monitoring and logging set up
- ☐ Backup and disaster recovery plan in place
- ☐ Load testing completed
- ☐ Security audit passed

Security Considerations

Robot Application Security

- Store API keys in Android Keystore, not hardcoded
- Use certificate pinning for backend communication
- Validate all input before sending to backend
- Implement rate limiting on user inputs
- Clear sensitive data from memory after session ends

Backend Security

- Implement API authentication (JWT or API keys)
- Use HTTPS/WSS for all communications
- Rate limiting on API endpoints
- Input sanitization to prevent injection attacks
- Regular security updates and patches
- Monitor for suspicious activity

Privacy Considerations

- Do not log personal user information
- Implement data retention policies
- Allow users to request data deletion
- Comply with GDPR, CCPA, or local privacy laws
- Provide privacy policy accessible from robot interface

Monitoring and Maintenance

Application Monitoring

- Log application errors to remote logging service
- Track key metrics:
 - Conversation success rate
 - Average response time
 - Voice recognition accuracy
 - Network connectivity uptime
 - Crash rate
- Set up alerts for critical failures
- Implement remote diagnostics capability

Backend Monitoring

- Monitor API response times
- Track RAG retrieval accuracy
- Monitor LLM token usage and costs
- Database query performance
- Server resource utilization (CPU, RAM, disk)
- Set up alerting for downtime or degraded performance

Success Criteria

The project will be considered successful when:

1. Robot successfully accepts both form and voice inputs
2. Voice recognition accuracy $\geq 90\%$ in quiet environment
3. Backend responds to queries within 2 seconds in 95% of cases
4. Both displays properly show synchronized content
5. RAG model provides relevant and accurate responses

- 6. System handles network failures gracefully
- 7. Application runs continuously without crashes for 24+ hours
- 8. User acceptance testing validates positive user experience

Risks and Mitigation

Risk	Probability	Mitigation Strategy
Keenon SDK documentation insufficient	Medium	Contact Keenon support early; reverse-engineer SDK if needed; allocate buffer time
Voice recognition poor in noisy environment	High	Implement noise cancellation; use directional microphones; provide touch fallback
RAG responses irrelevant or inaccurate	Medium	Curate high-quality knowledge base; fine-tune prompts; implement human feedback loop
Network latency/reliability issues	Medium	Implement local caching; optimize payload size; use WebSocket for efficiency
Display synchronization bugs	Low	Thorough testing; implement fallback single-display mode
LLM API cost overruns	Medium	Set usage limits; implement caching; consider local model for common queries

Table 4: Project Risk Assessment

Future Enhancements

Post-MVP features to consider:

1. Multi-language support beyond English and Hindi
2. Emotion detection from voice tone to adjust responses
3. Integration with robot navigation for guided delivery
4. Camera-based visual input (object recognition, QR code scanning)
5. Offline mode with local LLM for basic queries
6. Analytics dashboard for conversation insights
7. A/B testing framework for response optimization
8. Integration with external systems (POS, inventory management)

Support and Documentation

Developer Resources

- Keenon SDK Documentation: [Request from Keenon support]
- Android Development: <https://developer.android.com>
- LangChain Documentation: <https://docs.langchain.com>
- OpenAI API Reference: <https://platform.openai.com/docs>

Project Contacts

- Project Owner: Rishabh Rathi, Waaree Autonova
- Technical Lead: [To be assigned]
- Keenon Support: [Contact details from vendor]

Conclusion

This project represents an innovative integration of conversational AI with physical robotics hardware. By combining the Keenon robot platform with a custom-built RAG-powered backend, we aim to create an intelligent, interactive user experience that can handle complex queries through natural conversation.

Success depends on careful attention to the dual-input system (touch and voice), robust backend integration, and seamless dual-display management. The phased development approach allows for iterative

testing and refinement, ensuring each component works correctly before integration.

The developer should begin with Phase 1 foundation work, establishing basic communication between robot and backend. Once this core functionality is proven, input methods and display management can be layered on systematically.

Regular communication with the project owner and thorough documentation of decisions, challenges, and solutions will be essential for project success.

Appendix A: Sample Code Snippets

A.1: Voice Input Handler

```
public class VoiceInputManager {
    private SpeechRecognizer speechRecognizer;
    private Context context;

    public VoiceInputManager(Context context) {
        this.context = context;
        this.speechRecognizer = SpeechRecognizer.createSpeechRecognizer(context);

        speechRecognizer.setRecognitionListener(new RecognitionListener() {
            @Override
            public void onResults(Bundle results) {
                ArrayList<String> matches = results.getStringArrayList(
                    SpeechRecognizer.RESULTS_RECOGNITION
                );
                if (matches != null && !matches.isEmpty()) {
                    String recognizedText = matches.get(0);
                    handleRecognizedText(recognizedText);
                }
            }

            @Override
            public void onError(int error) {
                handleRecognitionError(error);
            }
        });
    }
}
```

```

    }

    // Other callback implementations...
  });
}

public void startListening() {
    Intent intent = new Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH);
    intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL,
        RecognizerIntent.LANGUAGE_MODEL_FREE_FORM);
    intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE, "en-US");

    speechRecognizer.startListening(intent);
}

private void handleRecognizedText(String text) {
    // Send to backend
}
}

```

A.2: Backend Client Interface

```

public interface BackendClient {
    void sendQuery(
        String sessionId,
        String query,
        List<ConversationMessage> history,
        BackendCallback callback
    );
}

```

```

    void createSession(String robotId, SessionCallback callback);

    void endSession(String sessionId);

    interface BackendCallback {
        void onSuccess(BackendResponse response);
        void onError(Exception e);
    }
}

```

```

    }

    interface SessionCallback {
        void onSessionCreated(String sessionId);
        void onError(Exception e);
    }
}

```

A.3: Display Manager

```

public class DualDisplayManager {
    private Display[] displays;
    private Presentation advertisingPresentation;

```

```

    public DualDisplayManager(Context context) {
        DisplayManager displayManager =
            (DisplayManager) context.getSystemService(Context.DISPLAY_SERVICE);
        displays = displayManager.getDisplays();

        // Find advertising display (non-touch, larger)
        for (Display display : displays) {
            if (isAdvertisingDisplay(display)) {
                advertisingPresentation = new AdvertisingPresentation(
                    context, display
                );
            }
        }
    }

    public void showOnAdvertisingDisplay(DisplayContent content) {
        if (advertisingPresentation != null) {
            advertisingPresentation.updateContent(content);
            advertisingPresentation.show();
        }
    }

    private boolean isAdvertisingDisplay(Display display) {

```

```
// Logic to identify 23.8" display
// May need to check display metrics
DisplayMetrics metrics = new DisplayMetrics();
display.getRealMetrics(metrics);
// Check diagonal size calculation
return metrics.widthPixels == 1920 && metrics.heightPixels == 1080;
}

}
```

References

- [1] Keenon Robotics. (2022). T2 Intelligent Transportation Robot Instruction Manual. *Keenon Robotics Co., Ltd.* <https://manuals.plus/keenon/t2-intelligent-transportation-robot-manual>
- [2] Lots of Bots. (2023). Keenon T10 Specifications V2.0. https://www.lotsofbots.com/media/robots/assets/Keenon_T10_V2.0-20230926.pdf
- [3] Viam Labs. (2024). Speech Service API - Voice Control for Robots. *GitHub Repository*. <https://github.com/viam-labs/speech-service-api>
- [4] Wang, L., et al. (2025). Human-robot interaction using retrieval-augmented generation and transformer neural networks. *PMC Journal*, 12336312. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12336312/>
- [5] Pichevin, D. (2025). AI Robot with OpenAI Realtime API - Conversational Robot Development. <https://www.davidpichevin.com/projects/ai-robot-with-openai-realtime-api>